ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN

Nguyễn Toàn Thắng
Hồ Minh Tiến
Dương Tùng Thiện
Đặng Thái Tú
(Nhóm sinh viên thực hiện)

THỰC HÀNH TRÍ TUỆ NHÂN TẠO NÂNG CAO

LAB 03: TÌM KIẾM CỤC BỘ

HILL CLIMBING - RANDOM RESTART - SIMULATED ANNEALING

MÔN HỌC: TRÍ TUỆ NHÂN TẠO NÂNG CAO

NGÀNH: CÔNG NGHỆ THÔNG TIN

GIẢNG VIÊN HƯỚNG DẪN:

TS. Đỗ Như Tài

Thành phố Hồ Chí Minh, năm 2026

Mục lục

1. MỤC TIÊU 3
2. BÀI TẬP THỰC HÀNH 4
2.1. Bài 1. Tìm kiếm cục bộ trên mê cung – Simple Hill Climbing 4
2.2. Bài 2. Mô hình hóa bài toán n quân hậu 4
2.3. Bài 3. Steepest-ascent và Stochastic Hill Climbing 5
2.4. Bài 4. Random Restart và Simulated Annealing 5
2.5. Bài 5. Khảo sát lịch giảm nhiệt độ 6
2.6. Bài 6. Tối ưu hàm đếm conflicts bằng Numba 7
2.7. Bài 7. Tìm kiếm cục bộ cho Traveling Salesman Problem 8
2.8. Bài 8. Tổng hợp thực nghiệm và phân tích 9
2.9. Câu hỏi thảo luận 10

Bảng phân công

Họ tên / MSSV	Công việc	Đánh giá mức độ hoàn thành (nhóm xác nhận)	Khối lượng (%)
Nguyễn Toàn Thắng 3123410343	WP3: Steepest-ascent; Efficient Stochastic; Random Restart. Discussion 1, 3, 4; tổng hợp báo cáo.	Đã có code và phần thảo luận; các số liệu một lần chạy và 100 lần chạy được trình bày theo nguồn.	25%
Hồ Minh Tiến 3123410373	Phần TSP: Swap; Steepest, Random Restarts, Stochastic, First-choice, SA và GA (bonus).	Notebook TSP và báo cáo thành viên đã có số liệu; cần xác nhận phân công cá nhân.	25%
Dương Tùng Thiện 3123410350	Đóng góp mô hình N-Queens/helper; nội dung thực nghiệm và khảo sát liên quan.	Có chú thích đóng góp trong notebook; phạm vi công việc cá nhân cần nhóm xác nhận.	25%
Đặng Thái Tú 3123410399	WP1: Maze; WP5: benchmark conflicts/Numba. Discussion 2, 8, 9; hỗ trợ phân tích n-Queens.	Có tài liệu Maze, benchmark và phần Discussion; đã tổng hợp.	25%

1. MỤC TIÊU

- Hiểu nguyên lý tìm kiếm cục bộ, hàm mục tiêu, phép sinh lân cận; phân biệt local optimum, plateau và ridge.
- Khảo sát Simple Hill Climbing, Steepest-ascent, Stochastic/First-choice, Random Restart và Simulated Annealing.
- Mô hình hóa, cài đặt và so sánh các thuật toán trên mê cung L, n-Queens và Traveling Salesman Problem (TSP).
- Đo chất lượng nghiệm, tỉ lệ thành công, chi phí đánh giá, thời gian và quá trình hội tụ; phân tích Numba/JIT và các lịch nhiệt.

Các bài thực hành được tổng hợp theo tám bài của đề Lab 03; phần thảo luận gồm đủ mười câu hỏi ở mục 4.3. Đơn vị trình bày là ms cho thời gian thuật toán (nếu có) và µs/bàn cho benchmark hàm conflicts. Những số liệu từ môi trường chạy hoặc các nhóm phép thử khác nhau không được dùng để suy ra thứ hạng tốc độ tuyệt đối.

Thuật toán	Quy tắc chọn trạng thái	Điểm cần quan sát
Simple Hill Climbing	Chọn bước cải thiện theo trật tự xét/biến thể chọn tốt nhất	Có thể mắc cực tiểu cục bộ
Steepest-ascent	Đánh giá mọi lân cận rồi chọn bước tốt nhất	Nhiều phép đánh giá
Stochastic / First-choice	Chọn ngẫu nhiên bước tốt / thử lân cận ngẫu nhiên	Đa dạng, phụ thuộc seed
Random Restart	Khởi động lại từ nhiều trạng thái	Tăng xác suất tìm nghiệm, tốn chi phí
Simulated Annealing	Nhận bước xấu với xác suất phụ thuộc T	Khám phá và khai thác phụ thuộc lịch nhiệt

Bảng 1. Tóm tắt các thuật toán theo mục tiêu của đề Lab 03.

2. BÀI TẬP THỰC HÀNH

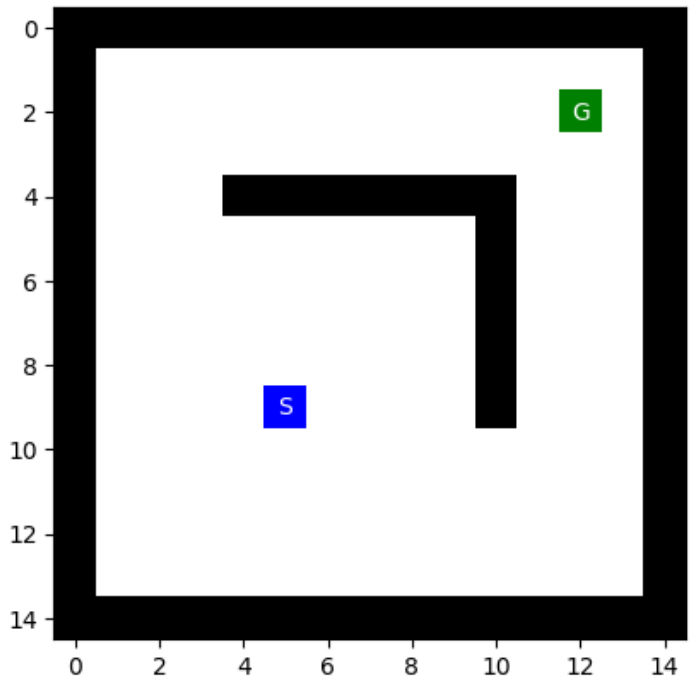
2.1. Bài 1. Tìm kiếm cục bộ trên mê cung – Simple Hill Climbing

Dữ liệu sử dụng là L_maze.txt (15 × 15), đọc bởi Maze_Example_LocalSearch.ipynb và maze_helper.py. Mỗi trạng thái là tọa độ (row, column), xuất phát tại S = (9, 5) và đích G = (2, 12). Hàng xóm hợp lệ sinh theo bốn hướng N, E, S, W; tường X được loại bỏ. Hàm đánh giá là khoảng cách Manhattan: h(r,c) = |r − 2| + |c − 12|, cần tối thiểu hóa.

Trong notebook, mỗi bước đánh giá tất cả hàng xóm và chọn trạng thái có h nhỏ nhất (phá hòa theo thứ tự duyệt); chỉ chấp nhận nếu h giảm nghiêm ngặt. Vì vậy dù notebook gọi local search/Hill Climbing, cách chọn là "best improvement" ở mỗi bước, không phải chọn ngay hàng xóm đầu tiên cải thiện.

Chỉ số	Quan sát
Giá trị ban đầu	h(S) = 14
Đường đi đã chấp nhận	(9,5) → (8,5) → (7,5) → (6,5) → (5,5) → (5,6) → (5,7) → (5,8) → (5,9)
Trạng thái dừng	(5,9), h = 6; tổng cộng 8 bước cải thiện
Đạt đích?	Không; result["solution"] là vị trí dừng, không phải G
Đổi thứ tự hướng	Thử 24 hoán vị, trường hợp L-maze đều kết thúc tại (5,9)

Bảng 2. Kết quả mê cung từ notebook và báo cáo WP1.



Hình 1. Bản đồ L-maze được hiển thị trong notebook mê cung.

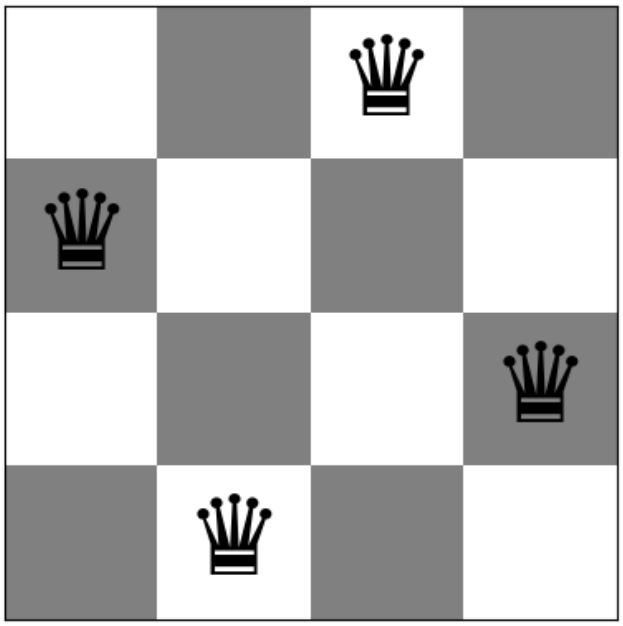
Tại (5,9), hai hướng bắc/đông bị chặn và hai trạng thái đi được (6,9), (5,8) đều có h = 7. Để đến đích phải tạm đi xa theo Manhattan nhằm vòng qua tường; do không chấp nhận bước xấu, agent dừng ở local optimum. Thứ tự N/E/S/W có thể đổi lựa chọn trung gian khi hòa, nhưng trong bản đồ đã thử không thay đổi vị trí dừng.

2.2. Bài 2. Mô hình hóa bài toán n quân hậu

Biểu diễn bàn cờ bằng vector b = [b_0, b_1, …, b_{n-1}] với b_j là chỉ số hàng của quân hậu tại cột j; mỗi cột có đúng một quân hậu. Không gian xét có n^n cấu hình thay vì chọn n ô bất kỳ từ n^2 ô.

Lân cận được tạo bằng cách chuyển một quân hậu sang một trong n − 1 hàng còn lại của cùng cột, có m = n(n − 1) lân cận ở mỗi trạng thái.

Hàm mục tiêu f(b) = conflicts(b) là số cặp quân hậu tấn công nhau theo hàng hoặc đường chéo; nghiệm hợp lệ khi f = 0. Với n = 4, cấu hình [1, 3, 0, 2] có 0 conflicts, là một lời giải được minh họa trong notebook.



Hình 2. Bàn cờ 4 quân hậu hợp lệ (0 conflicts) từ n_queens.ipynb.

2.3. Bài 3. Steepest-ascent và Stochastic Hill Climbing

Task 1 – Steepest-ascent: liệt kê n(n − 1) lân cận ở mỗi bước, chọn trạng thái có ít conflicts nhất nếu có cải thiện. Task 3 – Efficient Stochastic / First-choice: lấy mẫu một lân cận ngẫu nhiên, nhận ngay nếu tốt hơn; giới hạn số lượt thử không cải thiện để tránh lặp vô hạn. Task 2 – Inefficient Stochastic yêu cầu thu thập toàn bộ lân cận tốt trước khi chọn ngẫu nhiên, nhưng ô code Task 2 trong notebook được cung cấp vẫn đang trống.

Cấu hình	n	Conflicts cuối	Bước cải thiện	Lần gọi conflicts	Thời gian (ms)
Steepest-ascent	4	0	2	25	0,180
Efficient Stochastic	4	0	3	26	0,297
Steepest-ascent	8	1	3	225	1,296
Efficient Stochastic	8	1	6	331	3,285

Bảng 3. Kết quả một lần chạy trong n_queens.ipynb; không phải trung bình nhiều lượt.

Ở n = 8, cả hai lượt chạy đơn đều dừng ở conflicts = 1; kết quả không thể chứng minh một thuật toán tốt hơn trung bình. Với m = n(n − 1), Steepest và Inefficient Stochastic phải đánh giá đủ m láng giềng cho mỗi lượt quét; Efficient Stochastic có thể ít hơn nếu sớm gặp trạng thái cải thiện, nhưng có thể thử nhiều lần khi xác suất gặp bước tốt thấp.

2.4. Bài 4. Random Restart và Simulated Annealing

Random Restart lặp Hill Climbing trên các bàn ngẫu nhiên mới (tối đa 100 lần restart sau lần đầu, tức không quá 101 lần chạy) và giữ trạng thái tốt nhất. Một lượt minh họa của notebook cho n = 8 đạt kết quả như sau:

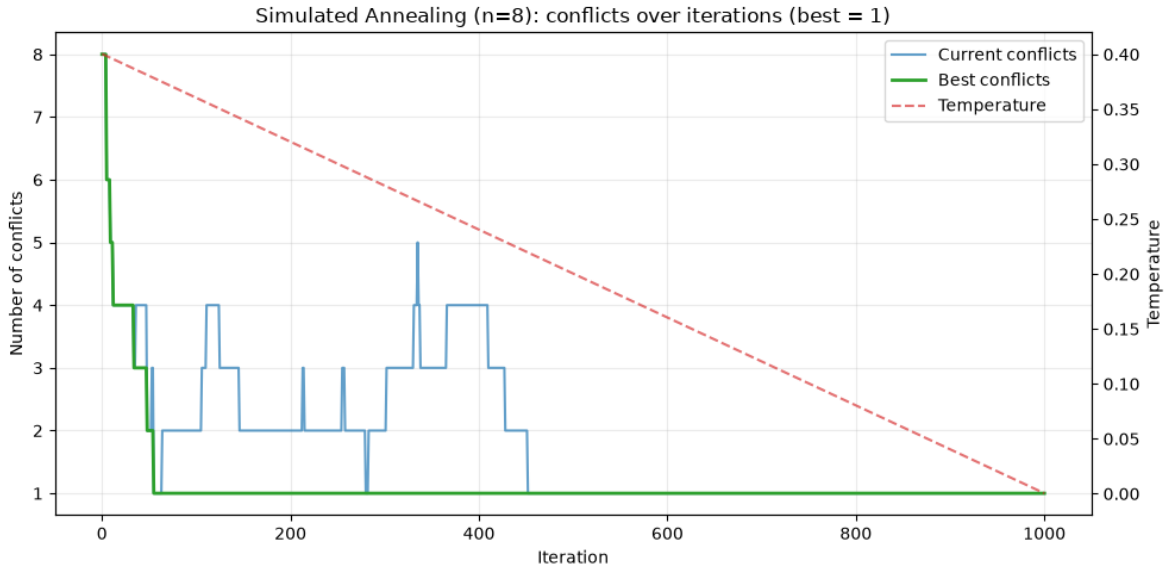
Thuật toán	Số lần chạy	Số restart	Evaluations	Conflicts cuối	Thời gian (ms)
Restart + Steepest	5	4	1.069	0	6,229
Restart + Efficient	4	3	1.115	0	11,102

Bảng 4. Minh họa Random Restart với n = 8 (một lượt từ notebook).

Simulated Annealing (SA) có xác suất chấp nhận lân cận xấu hơn $p = \exp(-\Delta/T)$ khi Δ = f(mới) − f(hiện tại) > 0; các bước không tăng conflicts luôn được nhận. Module simulated_annealing.py triển khai lịch tuyến tính hoặc hàm mũ, với mặc định $T_0 = 0{,}4$, giới hạn 1.000 vòng lặp, $T_{min} = 10^{-8}$. Mỗi bước chọn một cột và chuyển hậu đến hàng khác ngẫu nhiên.

Lượt chạy SA	Best conflicts	Số vòng lặp	Thời gian (ms)	Kết quả
n = 4 (đơn)	0	157	2,463	Thành công
n = 8 (đơn)	1	1.000	10,357	Chưa đạt f = 0
n = 4 (100 lượt)	Trung bình 0,00	Trung bình 38,30	TB 0,3495	100/100 đạt f = 0
n = 8 (100 lượt)	Trung bình 0,17	Trung bình 463,11	TB 4,4715	83/100 đạt f = 0

Bảng 5. SA: kết quả một lượt và tổng hợp 100 lượt theo output Task 5/6.



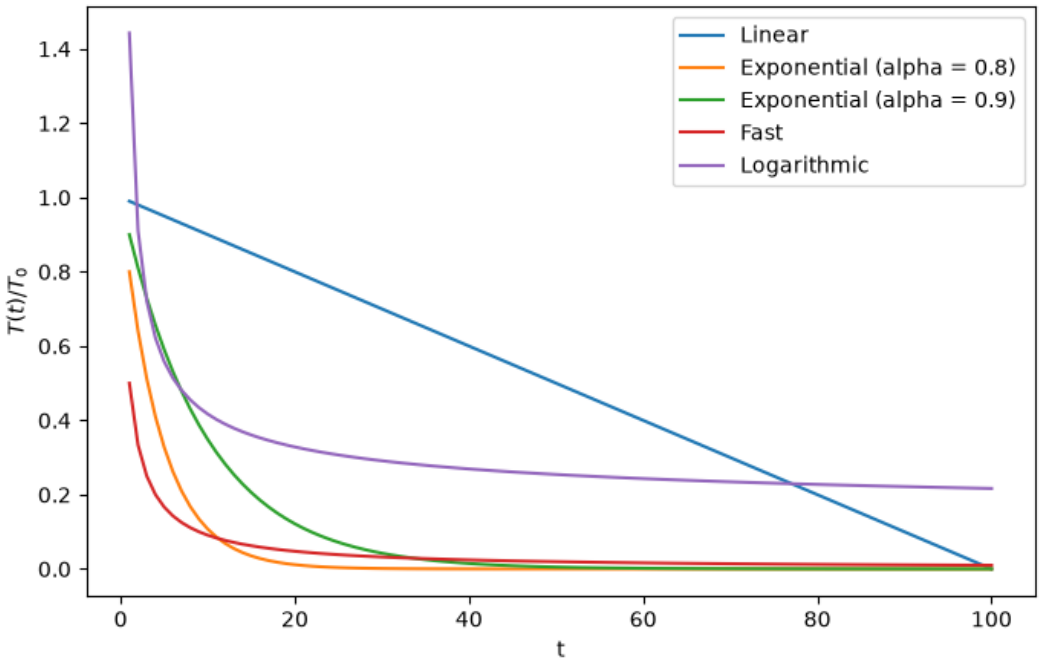
Hình 3. Số conflicts và nhiệt độ trong lượt SA n = 8 có trong notebook.

Lưu ý về tính lặp lại: cell Task 6 tạo 100 bàn đầu riêng biệt và truyền qua tham số board, nhưng hàm simulated_annealing() hiện ghi đè tham số này bằng random_board(n). Vì thế không thể coi kết quả 100 lượt là đánh giá trên đúng 100 bàn khởi tạo đã tạo ở cell; các lượt vẫn dùng bảng ngẫu nhiên được tạo nội bộ. Trường seed trong result cũng giữ hằng RANDOM_SEED = 1234 thay vì ghi lại seed riêng cho từng lần chạy. Đây là hạn chế cần sửa để benchmark công bằng/tái lập đúng như mô tả.

2.5. Bài 5. Khảo sát lịch giảm nhiệt độ

Lịch nhiệt	Công thức	Đặc điểm
Linear	$T(t) = T_0 - \eta t$	Giảm đều; dễ hết nhiệt sớm nếu η lớn
Exponential	$T(t) = T_0 \cdot \alpha^t$ $(0 < \alpha < 1)$	α càng gần 1 thì giữ nhiệt lâu hơn
Fast	$T(t) = T_0/(1+t)$	Giảm nhanh lúc đầu
Logarithmic	$T(t) = T_0/\ln(1+t)$, $t > 0$	Về dài hạn thường giảm chậm hơn một số lịch khác

Bảng 6. Bốn lịch nhiệt theo Simulated_Annealing_Schedules.ipynb.



Hình 4. So sánh T(t)/T₀ qua 100 bước: kết quả đồ thị notebook.

Với cùng điều kiện, notebook có minh họa một lần SA n = 4 theo hai lịch: Linear đạt f = 0 sau 78 vòng (0,690 ms) và Exponential đạt f = 0 sau 67 vòng (1,027 ms). Đây là hai lượt đơn, chưa đủ để khẳng định lịch nào tốt hơn về tỉ lệ thành công hay tốc độ. Lịch chậm/nhanh còn phụ thuộc α, η, T₀ và khoảng t khảo sát; chẳng hạn α = 0,995 có thể duy trì nhiệt tốt hơn logarithmic trong 100 bước.

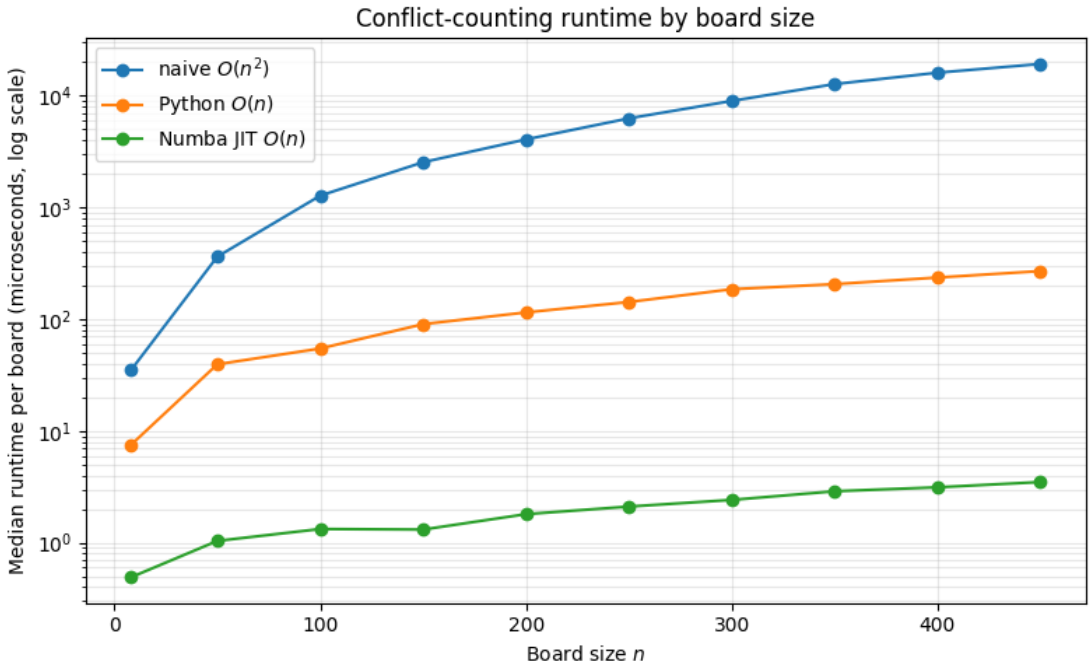
2.6. Bài 6. Tối ưu hàm đếm conflicts bằng Numba

conflicts_naive() duyệt từng cặp quân hậu nên tốn O(n²). Hàm conflicts() dùng mảng đếm cho hàng và hai loại đường chéo, sau đó cộng tổ hợp C(k,2) = k(k − 1)/2 đối với mỗi bộ đếm, đạt O(n) thời gian và O(n) bộ nhớ phụ. conflicts_jit() chạy cùng cách tính theo O(n) nhưng được Numba JIT biên dịch để giảm chi phí vòng lặp Python; JIT không làm thay đổi cấp độ phức tạp.

Trong notebook Numba, thử nghiệm dùng 20 bàn tạo trước mỗi n, 5 lần lặp phép đo, mỗi phép đo thực hiện 3 batch; số được in là median µs/bàn. Có warm-up conflicts_jit trước khi tính giờ và kiểm tra kết quả ba hàm trùng nhau trên các bàn test.

n	Naive O(n²) (µs)	Python O(n) (µs)	Numba JIT O(n) (µs)
8	34,962	7,495	0,490
50	363,213	39,518	1,038
100	1.276,802	54,703	1,327
150	2.543,545	90,403	1,312
200	4.069,185	115,152	1,802
250	6.281,367	142,725	2,107
300	8.993,040	185,787	2,417
350	12.731,312	206,282	2,888
400	16.093,910	236,228	3,133
450	19.273,372	269,882	3,493

Bảng 7. Median µs/bàn lấy trực tiếp từ output notebook Numba (khác batch của bản báo cáo WP5).



Hình 5. Thời gian hàm naive O(n²), Python O(n) và Numba JIT O(n).

Tại n = 450 trong batch này, bản Python O(n) mất 269,882 µs/bàn so với 19.273,372 µs/bàn của cách naive; Numba còn 3,493 µs/bàn. Báo cáo WP5 gửi kèm có một batch đo khác với các con số khác; không trộn hai batch trong cùng biểu đồ hoặc áp dụng chúng như cùng môi trường. Đặc biệt phải tách chi phí biên dịch JIT ở lần gọi đầu khỏi chi phí thực thi ổn định.

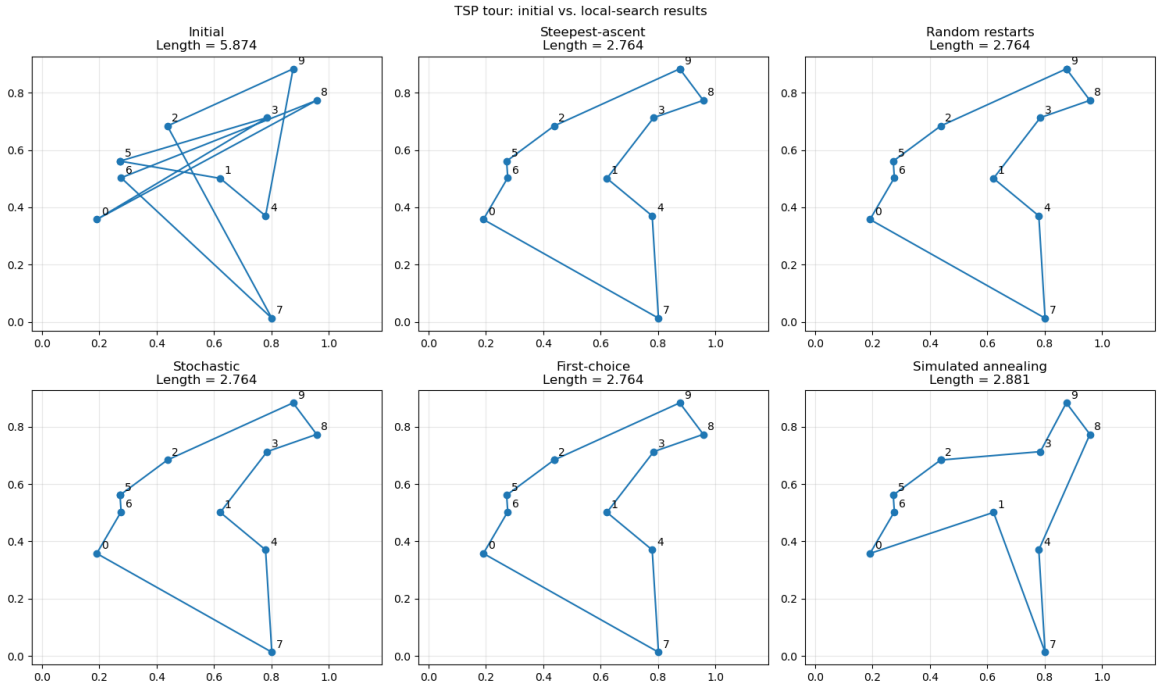
2.7. Bài 7. Tìm kiếm cục bộ cho Traveling Salesman Problem

Trạng thái là hoán vị n thành phố, trong bài sử dụng n = 10; hành trình phải trở lại thành phố xuất phát. Hàm mục tiêu L(π) là tổng khoảng cách Euclidean giữa các cặp thành phố liên tiếp cộng chặng cuối về đầu. Toán tử lân cận được dùng thống nhất là Swap: đổi hai vị trí i < j, tạo n(n − 1)/2 = 45 lân cận cho 10 thành phố.

Các thuật toán gồm Steepest-ascent; Steepest với 10 random restarts sau tour ban đầu (tối đa 11 lượt); Stochastic chọn ngẫu nhiên trong nhóm cải thiện; First-choice nhận cải thiện đầu tiên khi quét thứ tự ngẫu nhiên; SA với $T_0 = 1$, $\alpha = 0{,}995$, $T_{min} = 0{,}001$ và tối đa 5.000 vòng. Cùng một instance và tour xuất phát có $L_{initial} = 5{,}8739$ được dùng cho bảng sau.

Thuật toán	L cuối	ΔL giảm	Số lượt/bước	Evaluations	Thời gian (ms)
Steepest-ascent	2,76	3,11	7	360	2,04
Random Restarts	2,76	3,11	50	2.745	42,29
Stochastic	2,76	3,11	21	990	7,61
First-choice	2,76	3,11	12	111	2,01
Simulated Annealing	2,88	2,99	1.379	1.379	50,05

Bảng 8. Kết quả TSP một lần chạy, theo output notebook và báo cáo thành viên 4.



Hình 6. Hành trình ban đầu và các hành trình sau khi tối ưu trong notebook TSP.

Steepest, Random Restarts, Stochastic và First-choice cùng đạt độ dài 2,76 trong lượt ghi nhận; SA dừng ở 2,88. Không có chứng cứ xác nhận đây là giá trị tối ưu toàn cục, vì tất cả là các heuristic. Phần mở rộng Genetic Algorithm (bonus) đạt L = 2,7865 sau 200 thế hệ, với 9.850 lần đánh giá hàm mục tiêu, theo output đã lưu.

2.8. Bài 8. Tổng hợp thực nghiệm và phân tích

Kết quả tổng hợp lấy từ ba nhóm đầu ra: (1) Maze xác định, (2) N-Queens/SA từ notebook và báo cáo WP1/WP3, (3) TSP từ notebook/báo cáo thành viên. Các bài toán có hàm mục tiêu và giới hạn vòng lặp khác nhau nên không so sánh trực tiếp thời gian giữa chúng. Riêng N-Queens, dữ liệu thử nghiệm 100 lượt là cơ sở xem xét biến thiên kết quả trong từng cấu hình.

Thuật toán (100 lần mỗi cấu hình)	n	Tỉ lệ f = 0	Conflicts cuối TB	Bước cải thiện TB	Thời gian TB (ms)
Steepest-ascent	4	46%	0,58	1,51	0,086
Efficient Stochastic	4	41%	0,70	1,80	0,395
Restart + Steepest	4	100%	0,00	3,70	0,259
Restart + Efficient	4	100%	0,00	5,93	1,168
Steepest-ascent	8	18%	1,18	3,34	1,368
Efficient Stochastic	8	18%	1,25	4,91	2,457
Restart + Steepest	8	100%	0,00	24,93	21,271
Restart + Efficient	8	100%	0,00	32,73	138,800

Bảng 9. Batch 100 lần chạy mỗi cấu hình từ báo cáo WP1/WP5 của Đặng Thái Tú; điều kiện benchmark của batch này khác SA trong notebook.

Random Restart có 100% lần thử thành công ở n = 4 và n = 8 trong batch ghi nhận, cao hơn chạy đơn; tuy nhiên không chứng minh đảm bảo tuyệt đối. Cần ngân sách nhiều lượt và thường tốn thêm thời gian/đánh giá. Với SA, notebook ghi nhận 100% (n = 4), 83% (n = 8) trong batch 100 lượt riêng. Chỉ có kết quả tìm nghiệm ở n = 4 và 8, không đủ để suy diễn thời gian giải N-Queens với n = 12, 16 hoặc 20.

Về hội tụ, các lượt Steepest và First-choice minh họa giảm conflicts nhanh đầu quá trình rồi dừng tại trạng thái không còn/khó gặp cải thiện. SA có thể tạm tăng conflicts và giữ best-so-far theo thời gian. Tuy nhiên, cell "Algorithm Convergence" của Task 6 đang trống: chưa có đồ thị hội tụ so sánh đầy đủ mọi thuật toán. Cell "Problem Size Scalability" cũng chưa có đồ thị log-log n = 4, 8, 12, 16, 20 cho hai thuật toán như đề yêu cầu. Đồ thị benchmark conflicts O(n)/O(n^2) ở Bài 6 không thay thế đo khả năng mở rộng của thuật toán giải.

2.9. Câu hỏi thảo luận

Câu 1. Tìm kiếm cục bộ khác BFS, A* về mục tiêu và dữ liệu cần lưu trữ như thế nào?

BFS/A* hướng tới đường đi từ trạng thái đầu tới đích; BFS tối ưu số bước khi chi phí cạnh như nhau, A* tối ưu nếu heuristic và cơ chế quản lý trạng thái phù hợp. Local Search nhằm tìm một trạng thái có giá trị hàm mục tiêu tốt, không nhất thiết xây dựng chuỗi hành động tối ưu để đến đó.

BFS thường quản lý frontier, reached và thông tin cha; A* còn lưu g(n), h(n), f(n) và cấu trúc ưu tiên. Hill Climbing chủ yếu giữ trạng thái đang xét và các trạng thái lân cận; thường ít dữ liệu hơn nhưng dễ mắc cực trị cục bộ. Ví dụ n-Queens quan tâm f(board) = 0, không phải chiều dài đường đi của các lần di chuyển quân hậu.

Câu 2. Vì sao Hill Climbing có thể mắc kẹt ở local optimum, plateau hoặc ridge?

Local optimum là trạng thái tốt hơn các láng giềng nhưng chưa chắc tốt nhất toàn cục; plateau có các trạng thái ngang điểm khiến thuật toán thiếu hướng cải thiện; ridge yêu cầu chuỗi bước phối hợp hoặc bước tạm xấu hơn. Quy tắc chỉ nhận cải thiện khiến Hill Climbing khó vượt các vùng này.

Ví dụ L-maze dừng tại (5,9), h = 6 vì muốn đi vòng qua tường phải tạm làm tăng khoảng cách Manhattan. Các hướng khắc phục gồm restart, cho phép sideways moves có giới hạn, dùng lân cận khác hoặc SA.

Câu 3. So sánh Steepest-ascent, Stochastic (liệt kê mọi bước tốt) và First-choice theo số lần đánh giá.

Với n-Queens, có m = n(n − 1) lân cận. Steepest và Inefficient Stochastic đều phải duyệt toàn bộ m láng giềng mỗi lượt để chọn tốt nhất hoặc bốc ngẫu nhiên trong nhóm cải thiện. First-choice sinh thử ngẫu nhiên cho đến khi gặp nước tốt nên có thể đánh giá ít hơn m nếu bước tốt phổ biến.

Nếu xác suất gặp bước tốt trong một lần thử là q > 0, số thử kỳ vọng gần 1/q theo mô hình lấy mẫu độc lập. Khi q rất nhỏ, First-choice có thể tốn nhiều thử; giới hạn 5n(n − 1) phép thử không cải thiện liên tiếp trong code WP3 là điều kiện dừng thực nghiệm, không chứng minh cực trị cục bộ. Task 2 vẫn cần cài đặt để kiểm chứng phép so sánh đủ cả ba.

Câu 4. Random Restart cải thiện xác suất tìm lời giải như thế nào? Khi nào không còn hiệu quả?

Nếu một lượt độc lập có xác suất thành công p, k lượt độc lập thành công ít nhất một lần với xác suất 1 − (1 − p)^k. Ví dụ giả định p = 0,2 thì sau 5 lượt là 67,23%, sau 10 lượt là 89,26%; đây là ví dụ lý thuyết, không phải xác suất thực đo của nhóm.

Restart thêm ít hiệu quả khi xác suất tích lũy đã cao, chi phí mỗi lượt vượt lợi ích hoặc xác suất tìm nghiệm từ mỗi khởi tạo gần 0 vì hàm mục tiêu/toán tử kém phù hợp. Lượt n = 8 trong WP3 cần 4 restart/1.069 evaluations cho Steepest và 3 restart/1.115 evaluations cho Efficient, cho thấy ít restart không đồng nghĩa ít đánh giá.

Câu 5. Vì sao Simulated Annealing chấp nhận nghiệm xấu hơn? Tác động nhiệt độ tới khám phá và khai thác?

Khi $\Delta > 0$, SA nhận trạng thái xấu hơn với xác suất $\exp(-\Delta/T)$ để có cơ hội vượt rào cản cục bộ. Nhiệt độ cao cho xác suất lớn, tăng exploration; nhiệt độ thấp khiến thuật toán thiên về exploitation (giữ hoặc cải thiện nghiệm). Với $\Delta \leq 0$, thuật toán chấp nhận luôn.

Khả năng tìm nghiệm phụ thuộc lịch nhiệt, thời gian chạy và cấu trúc bài toán; không đảm bảo tối ưu toàn cục trong số vòng giới hạn. Kết quả SA n = 8 trong notebook cho thấy một lượt dừng ở conflicts = 1 sau 1.000 vòng.

Câu 6. Lịch Linear, Exponential, Fast và Logarithmic khác nhau? Lịch nào phù hợp thí nghiệm?

Linear: $T_0 - \eta t$ (giảm đều); Exponential: $T_0\alpha^t$ (phụ thuộc mạnh α); Fast: $T_0/(1+t)$ (giảm nhanh đầu quá trình); Logarithmic: $T_0/\ln(1+t)$, $t > 0$ (giảm chậm về dài hạn). Tuy nhiên thứ tự "lịch nào giữ nhiệt lâu nhất" phụ thuộc tham số và khoảng quan sát.

Nhóm sử dụng Linear làm cấu hình cơ sở và so với Exponential trong SA n-Queens. Hai lượt đơn n = 4 đều đạt f = 0 nhưng số bước/thời gian khác nhau; cần lặp lại trên cùng ngân sách và dữ liệu mới xác định lịch phù hợp hơn.

Câu 7. Vì sao biểu diễn n-Queens mỗi cột đúng một quân giúp giảm không gian tìm kiếm?

Khi đặt đúng một quân hậu ở mỗi cột, trạng thái chỉ còn vector độ dài n: mỗi phần tử có n lựa chọn hàng, tổng cộng n^n cấu hình. Ví dụ n = 4 có $4^4 = 256$ cấu hình, ít hơn $C(16,4) = 1.820$ cách chọn bốn ô bất kỳ trên bàn 4×4.

Biểu diễn tự loại xung đột cùng cột và làm phép sinh lân cận đơn giản: thay hàng của một quân tại một cột. Xung đột ngang và chéo vẫn phải tính bằng conflicts().

Câu 8. Vì sao đếm xung đột theo hàng và đường chéo có độ phức tạp O(n) thay vì $O(n^2)$?

Mỗi quân hậu đóng góp một lần vào bộ đếm của hàng r, chéo r+c và chéo r−c+n. Với mỗi bộ đếm chứa k quân hậu, số cặp xung đột là $C(k,2) = k(k-1)/2$. Việc duyệt n quân hậu và duyệt các mảng đếm có kích thước O(n) nên tổng chi phí là O(n), bộ nhớ bổ sung O(n).

Cách naive kiểm tra từng cặp quân hậu nên thực hiện $n(n-1)/2$ so sánh, $O(n^2)$. Numba JIT chỉ giảm overhead thực thi cho O(n), không làm thay đổi độ phức tạp.

Câu 9. Khi dùng Numba, vì sao phải loại trừ lần biên dịch JIT đầu khỏi benchmark?

Lần gọi đầu có thể kích hoạt biên dịch mã Python sang mã máy và thiết lập chữ ký kiểu dữ liệu; đây là một chi phí một lần, không phải thời gian chạy ổn định của hàm đã biên dịch. Nếu tính vào benchmark sẽ phóng đại thời gian Numba, nhất là với hàm nhỏ có thời gian thực thi chỉ vài µs.

Do đó notebook gọi conflicts_jit(random_board(8)) trước đo, dùng cùng kiểu dữ liệu và benchmark các lượt sau. Số liệu median µs/bàn được trình bày ở Bảng 7.

Câu 10. Với TSP, đổi toán tử Swap sang 2-opt ảnh hưởng thế nào đến lời giải và số láng giềng?

Swap đổi vị trí hai thành phố bất kỳ, bảo toàn tính hợp lệ của hoán vị; với n thành phố có $n(n-1)/2$ cặp hoán đổi. 2-opt thay hai cạnh và đảo ngược một đoạn hành trình, cũng bảo toàn một tour; thao tác này thường tháo các cạnh giao nhau và có thể tạo các cải thiện mà Swap phải đi qua nhiều bước.

Cả Swap và 2-opt đều có kích thước lân cận bậc $O(n^2)$, nhưng tập láng giềng và đường đi trên không gian trạng thái khác nhau; khi cố định điểm xuất phát và loại trường hợp tương đương, số phép 2-opt duy nhất có thể ít hơn số cặp Swap. Notebook hiện dùng Swap; chưa có kết quả thực nghiệm chạy 2-opt nên không thể định lượng độ cải thiện hay chênh lệch thời gian.

3. NHẬN XÉT VÀ KẾT LUẬN

Tính chất thuật toán và cực trị cục bộ. Mê cung L cho thấy trực quan thất bại của quy tắc chọn cải thiện nghiêm ngặt: agent giảm Manhattan liên tục từ 14 đến 6 nhưng không tới G. Với n-Queens và TSP, các thuật toán đánh giá trạng thái cục bộ có thể sớm tìm cấu hình khá tốt song dừng trước nghiệm tối ưu. Thay đổi thứ tự, nguồn ngẫu nhiên hoặc toán tử lân cận có thể thay đổi tiến trình, nhưng không tự đảm bảo tối ưu toàn cục.

Random Restart và SA. Khởi động lại mở rộng số vùng tìm kiếm, do đó cải thiện đáng kể tỉ lệ thành công trong bộ số liệu n-Queens n = 4 và n = 8 của WP1. Đổi lại số lần đánh giá và thời gian tăng. SA tạo cơ hội rời cực tiểu cục bộ nhờ chấp nhận bước xấu, với tỉ lệ tìm được f = 0 thay đổi khi n tăng; thí nghiệm SA 100 lượt ghi nhận 83% ở n = 8, trong khi n = 4 là 100%. Những tỷ lệ này chỉ là quan sát của batch đã lưu, không phải cam kết hiệu năng.

Lịch nhiệt. Không có một schedule tốt tuyệt đối. Linear dễ dự đoán ngân sách nhiệt, Exponential điều chỉnh được tốc độ thông qua α, còn Fast và Logarithmic cho quỹ đạo nhiệt khác biệt. So sánh công bằng đòi hỏi thống nhất random seed, cùng dữ liệu, số vòng lặp và điều kiện dừng. Chỉ với một lượt đơn theo từng lịch không nên kết luận lịch nào "nhanh nhất".

Hiệu quả tính toán. Benchmark hàm conflicts cho thấy chuyển từ so sánh từng cặp $O(n^2)$ sang đếm theo hàng và đường chéo $O(n)$ có lợi rõ khi n lớn; Numba tiếp tục giảm chi phí thực thi. Tuy nhiên phép đo hàm đánh giá không cho biết tự động thuật toán Hill Climbing hay SA sẽ mở rộng tốt đến n = 20; phải đo riêng vòng tìm kiếm, số evaluations và tỷ lệ nghiệm.

TSP. Với toán tử Swap và một bộ 10 thành phố, bốn biến thể Hill Climbing/Restart đạt L ≈ 2,76 trong lượt so sánh; SA đạt 2,88. Số liệu này cho thấy đánh đổi giữa chi phí và chất lượng của lần chạy nhưng không chứng minh ưu thế thống kê hay tối ưu toàn cục. Kết quả GA bonus bổ sung góc nhìn tiến hóa, cần so sánh trong cùng ngân sách đánh giá nếu muốn xếp hạng công bằng.

Hạn chế báo cáo. Bốn nguồn thành viên có phạm vi và điều kiện đo không hoàn toàn trùng nhau; WP5 có benchmark độc lập khác số với output notebook. Task 2, đường hội tụ liên thuật toán và log-log scalability trong n_queens.ipynb đang trống. Thêm nữa hàm SA chưa sử dụng tham số board đầu vào và trường seed chưa đại diện seed độc lập từng lượt. Vì vậy báo cáo giữ nguyên giới hạn chứng cứ, tránh gọi các mục trên là hoàn thành hoặc suy ra hiệu quả chưa được đo.

Kết luận. Nhóm đã thể hiện được điểm mạnh/yếu của các chiến lược tìm kiếm cục bộ qua ba bài toán thực hành; nắm được cách cải thiện khả năng thoát cực trị bằng Restart/SA và sự khác nhau giữa tối ưu hóa thuật toán O(n) với tăng tốc triển khai JIT. Trước khi nộp chính thức, nhóm cần hoàn thiện các cell còn thiếu, sửa khả năng tái lập SA, chạy thí nghiệm bổ sung và chốt bảng phân công/MSSV.

4. TÀI LIỆU SỬ DỤNG TRONG BÀI THỰC HÀNH

- TTNTNC - Lab03 - Tim kiem cuc bo.pdf: yêu cầu 8 bài tập, 10 câu Discussion và danh mục sản phẩm nộp.
- Maze_Example_LocalSearch.ipynb; L_maze.txt; maze_helper.py: minh họa mê cung và local optimum.
- n_queens.ipynb; helper.py; simulated_annealing.py: mô hình, Hill Climbing, Random Restart, SA và đầu ra thực nghiệm.
- Simulated_Annealing_Schedules.ipynb; annealing_schedules.py: bốn lịch giảm nhiệt và kiểm tra bằng đồ thị.
- n_queens_fast_conflict_calculation_with_numba.ipynb: phép đo naive/Python O(n)/Numba JIT.
- traveling_salesman_problem.ipynb: mô hình TSP, các thuật toán và hình tour; GA bonus.
- ThanhVien1.txt; ThanhVien2.docx; ThanhVien3.md; ThangVien4.pdf; Lab03_LocalSearch_ThaiTu.docx: nguồn nội dung và số liệu các thành viên gửi.
- Bao_cao.pdf: mẫu Lab 02 sử dụng để mô phỏng hình thức trang bìa, mục lục, bảng phân công và bố cục.

5. DANH MỤC TÀI LIỆU THAM KHẢO

[1] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed., Pearson, 2021, chương tìm kiếm.

[2] M. Hahsler, Introduction to Artificial Intelligence – Course Repository, https://github.com/mhahsler/Introduction_to_Artificial_Intelligence (notebook nguồn).

[3] S. Kirkpatrick, C. D. Gelatt Jr., and M. P. Vecchi, “Optimization by simulated annealing,” Science, vol. 220, pp. 671–680, 1983.

[4] B. Hajek, “Cooling schedules for optimal annealing,” Mathematics of Operations Research, 1988.

[5] Tài liệu giảng viên: “TTNTNC – Lab03 – Tìm kiếm cục bộ”, Trường Đại học Sài Gòn, 2026.

PHỤ LỤC A. ĐỐI CHIẾU TIẾN ĐỘ VÀ GIỚI HẠN

Hạng mục theo đề	Đối chiếu từ file đã nhận	Trạng thái cần lưu ý
Bài 1 - Maze	Đường đi, vị trí dừng, phân tích Manhattan và thứ tự hướng có trong WP1/notebook.	Đã có minh họa và giải thích.
Bài 2 - N-Queens	Vector trạng thái, số lân cận, conflicts và bàn n = 4, 8.	Đã mô hình hóa.
Bài 3 - HC	Task 1/3 đã có mã và đầu ra.	Task 2 còn ô trống.
Bài 4 - Restart/SA	Restart Task 4 và module SA Task 5 có đầu ra.	SA bỏ qua tham số board truyền vào; seed log chưa đúng từng lượt.
Bài 5 - Schedules	4 lịch và đồ thị; hai lượt Linear/Exponential.	Chưa có so sánh xác suất/thời gian nhiều lượt giữa các lịch.
Bài 6 - Numba	3 hàm, median benchmark và 2 đồ thị.	Batch đo WP5 và notebook khác nhau; không gộp.
Bài 7 - TSP	Các thuật toán và hình tour; bonus GA.	Bảng hiệu năng 1 lượt; chưa có phân bố nhiều seed.
Bài 8 - Analysis	N-Queens n = 4, 8 có 100 lượt từ WP1; SA 100 lượt trong notebook.	Chưa có convergence đầy đủ, n = 12/16/20, log-log plot.
Discussion 1–10	Đã tổng hợp nội dung của bốn thành viên và diễn giải theo notebook.	Câu 10 về 2-opt mới ở mức phân tích, chưa cài đặt đo.
Thông tin nhóm	Đã giữ tên/MSSV có trong mẫu Lab 02.	Thắng, Tú cần bổ sung MSSV; đóng góp 25% tạm, cần xác nhận.

Bảng A1. Những công việc cần kiểm tra trước khi bàn giao notebook và nộp báo cáo.

Nguyên tắc sử dụng số liệu: bản báo cáo này không chạy lại toàn bộ notebook. Các thời gian, kết quả thực nghiệm và hình có chú thích được trích từ outputs của notebook/tài liệu nhóm cung cấp. Mọi nghiên cứu kết luận về so sánh tốc độ nên bổ sung chạy lại tập trung bằng cùng phiên bản Python, môi trường ảo, máy và danh sách seed; đo median/IQR thay vì chỉ một lượt.